

UNIT-2

Understanding Data – 2: Mathematics for Multivariate Data, Feature Engineering and Dimensionality Reduction Techniques.

Basic Learning Theory: Design of Learning System, Introduction to Concept of Learning, Induction biases, Modelling in Machine Learning , learning Frameworks.

- **Pedagogy/Course Delivery tools:** Chalk and talk, PowerPoint Presentation, Videos
- **Links:** <https://nptel.ac.in/courses/106106198>
<https://www.youtube.com/watch?v=TIFFfLejkcA>

**Topics to be
covered in
UNIT-2**

Understanding Data – 2:

2.8 Mathematics for Multivariate Data

2.8.1 Linear Systems and Gaussian Elimination for Multivariate Data

2.8.2 Matrix Decompositions

2.8.3 Machine Learning and Importance of Probability and Statistics

Probability Distributions

Density Estimation

2.10 Feature Engineering and Dimensionality Reduction
Techniques

Basic Learning Theory:

3.3 Design of Learning System

3.4 Introduction to Concept of Learning Induction biases

Modelling in Machine Learning

learning Frameworks

2.8 ESSENTIAL MATHEMATICS FOR MULTIVARIATE DATA.

Mathematical foundations required for handling multivariate data in machine learning:

- Machine learning relies heavily on linear algebra, statistics, probability, and information theory.
- Linear algebra serves as the fundamental language of data. It handles key components such as linear equations, vectors, matrices, vector spaces, and transformations, which form the bedrock of machine learning algorithms.

Mathematical concept

Vectors

Matrices

Mean vector

Variance

Covariance

Covariance matrix

Correlation

Standardization

Distance

Dot product

Matrix operations

Determinant & inverse

Eigenvalues/eigenvectors

Probability distributions

Multivariate normal distribution

Importance in multivariate data

Represent individual observations

Represent complete datasets

Represents multivariate center

Measures individual-variable spread

Measures joint variation

Represents variance and covariance relationships

Measures standardized linear relationships

Places variables on comparable scales

Measures similarity/dissimilarity

Used for projections and linear models

Support multivariate calculations

Important for matrix-based methods

Fundamental to PCA

Model uncertainty and relationships

Important theoretical distribution

2.8.1 Linear Systems and Gaussian Elimination for Multivariate Data

- If there is a unique solution, then the system is called consistent independent. If there are various solutions, then the system is called consistent dependant. If there are no solutions and if the equations are contradictory, then the system is called inconsistent.
- For solving large number of system of equations, **Gaussian elimination** can be used. The procedure for applying Gaussian elimination is given as follows:

Gaussian elimination is a systematic method for solving a system of linear equations.

The basic idea is to transform the equations into an **upper triangular form** and then solve them using **back substitution**.

Steps

1. Write the equations as an augmented matrix.
2. Select a pivot element.
3. Eliminate the variables below the pivot.
4. Continue for the remaining rows.
5. Obtain an upper triangular matrix.
6. Use back substitution to find the unknowns.

To facilitate the application of Gaussian elimination method, the following row operations are applied:

1. Swapping the rows
2. Multiplying or dividing a row by a constant
3. Replacing a row by adding or subtracting a multiple of another row to it

Example of Gaussian Elimination

Consider:

$$2x + y + z = 10$$

$$4x - 6y + 3z = 12$$

$$-2x + 7y + 2z = 5$$

Write the augmented matrix

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 10 \\ 4 & -6 & 3 & 12 \\ -2 & 7 & 2 & 5 \end{array} \right]$$

Eliminate x

$$R_2 \rightarrow R_2 - 2R_1$$

$$R_3 \rightarrow R_3 + R_1$$

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 10 \\ 0 & -8 & 1 & -8 \\ 0 & 8 & 3 & 15 \end{array} \right]$$

Eliminate y

$$R_3 \rightarrow R_3 + R_2$$

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 10 \\ 0 & -8 & 1 & -8 \\ 0 & 0 & 4 & 7 \end{array} \right]$$

Now the matrix is in upper triangular form.

Back Substitution

From the third equation:

$$4z = 7 \quad z = \frac{7}{4} = 1.75$$

From the second equation:

$$-8y + z = -8$$

Substitute $z=1.75$

$$-8y + 1.75 = -8$$

$$-8y = -9.75$$

$$y = 1.21875$$

From the first equation:

$$2x + y + z = 10$$

$$2x + 1.21875 + 1.75 = 10$$

$$2x = 7.03125$$

$$x = 3.515625$$

Thus, the solution is approximately:

$$x = 3.516, \quad y = 1.219, \quad z = 1.750$$

Example 2.8: Solve the following set of equations using Gaussian Elimination method.

$$2x_1 + 4x_2 = 6$$

$$4x_1 + 3x_2 = 7$$

Solution: Rewrite this in matrix form as follows:

$$\begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} \\ \sim \begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} R_1 = \frac{R_1}{2}$$

Apply the transformation by dividing the row 1 by 2. There are no general guidelines of row operations other than reducing the given matrix to row echelon form. The operator $\sim$ means reducing to. The above matrix can further be reduced as follows:

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 4 & 3 & | & 7 \end{pmatrix} R_2 = R_2 - 4R_1 \\ \sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & -5 & | & -5 \end{pmatrix} R_2 = R_2 / -5 \\ \sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & 1 & | & 1 \end{pmatrix} R_1 = R_1 - 2R_2 \\ \sim \begin{pmatrix} 1 & 0 & | & 1 \\ 0 & 1 & | & 1 \end{pmatrix}$$

Therefore, in the reduced echelon form, it can be observed that:

$$x_2 = 1$$

$$x_1 = 1$$

2.8.2 Matrix Decompositions

Matrix decomposition means breaking a matrix into two or more simpler matrices. These simpler matrices make it easier to solve mathematical and computational problems.

One important decomposition is **LU decomposition**.

Example 2.9: Find LU decomposition of the given matrix:

$$A = \begin{pmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{pmatrix}$$

Solution: First, augment an identity matrix and apply Gaussian elimination. The steps are as shown in:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 2 & 4 \\ 0 & 1 & 0 & 3 & 3 & 2 \\ 0 & 0 & 1 & 3 & 4 & 2 \end{array} \right] \quad \boxed{\text{Initial Matrix}}$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 2 & 4 \\ 3 & 1 & 0 & 0 & -3 & -10 \\ 0 & 0 & 1 & 3 & 4 & 2 \end{array} \right] \quad \boxed{R_2 = R_2 - 3R_1}$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 2 & 4 \\ 3 & 1 & 0 & 0 & -3 & -10 \\ 3 & 0 & 1 & 0 & -2 & -10 \end{array} \right] \quad \boxed{R_3 = R_3 - 3R_1}$$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 2 & 4 \\ 3 & 1 & 0 & 0 & -3 & -10 \\ 3 & \frac{2}{3} & 1 & 0 & 0 & -\frac{10}{3} \end{array} \right] \quad \boxed{R_3 = R_3 - \frac{2}{3}R_2}$$

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{pmatrix} \text{ and } U = \begin{pmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{pmatrix}.$$

It can be cross verified that the multiplication of LU yields the original matrix A

2.8.3 Machine Learning and Importance of Probability and Statistics

Statistics and probability form a core foundation of machine learning, serving an essential role alongside linear algebra to enable effective data analysis.

- **Data Modeling & Probability Distributions:** Datasets in machine learning are viewed as outcomes of probability distributions. Because real-world data often stems from various overlapping distributions, understanding random variables and probability distributions is crucial for mastering machine learning models.
- **Hypothesis and Model Evaluation:** Machine learning relies heavily on statistical concepts to formulate hypotheses, construct algorithms, assess model performance through hypothesis testing, and measure statistical significance.
- **Dataset Construction:** Sampling theory plays a vital role in designing and building representative datasets for training and testing models.

Probability Distributions

A probability distribution summarizes the probabilities associated with a variable's (say X) events, acting as a parameterized mathematical function that describes relationships between observations in a sample space. Data is said to follow a distribution if it obeys the mathematical function characterizing it, which allows calculating the probability of individual observations.

Types of Probability Distributions

1. Discrete probability distribution
2. Continuous probability distribution

Probability Density Function (PDF): Summarizes continuous probability distributions, calculates the probability of observing an instance, and displays the shape of the distribution when plotted.

Cumulative Distributive Function (CDF): Computes the probability of an observation being less than or equal to a value x . It is also defined by calculating the area under the curve for a small interval around a specific outcome.

- Both PDF and CDF yield continuous values.
- The discrete equivalent of PDF in a discrete distribution is called the **Probability Mass Function (PMF)**.

Continuous distribution:

A **continuous distribution** describes a random variable whose possible outcomes fall along a continuous spectrum or interval of real numbers (such as height, weight, time, or temperature), rather than isolated points.

Unlike discrete variables, the probability of a continuous random variable taking on an exact single value is always zero ($P(X=c)=0$). Instead, probabilities are calculated for intervals of values (e.g., $P(a \leq X \leq b)$) by finding the area under a curve.

- **Probability Density Function (PDF):** Denoted as $f(x)$, it describes the relative likelihood of a random variable taking on a value near x . The height of the curve does not represent probability directly—the area under the curve does.
- **Cumulative Distribution Function (CDF):** Denoted as $F(x)=P(X \leq x)$, it measures the total accumulated probability up to x :

$$F(x) = \int_{-\infty}^x f(t) dt$$

Rules

- **Non-negativity:** $f(x) \geq 0$ for all real numbers x .
- **Total Area Equals 1:** The total area under the PDF curve across its entire domain is exactly 1:

$$\int_{-\infty}^{\infty} f(x) dx = 1$$

- **Interval Probabilities:** The probability that X falls between a and b is given by the definite integral:

$$P(a \leq X \leq b) = \int_a^b f(x) dx = F(b) - F(a)$$

A **continuous probability distribution** is used when a random variable can take **any value within a range**, including decimal values.

Example: Student's Height

Suppose the height of students in a class is normally distributed with:

- Mean height = **165 cm**
- Standard deviation = **5 cm**

Let X = height of a randomly selected student.

The probability distribution can be represented as:

$$X \sim N(165, 5^2)$$

where N represents the **Normal Distribution**.

For example, we may want to find:

$$P(160 \leq X \leq 170)$$

This means:

What is the probability that a randomly selected student's height is between **160 cm and 170 cm**?

The probability is obtained by finding the **area under the normal distribution curve** between 160 and 170 cm.

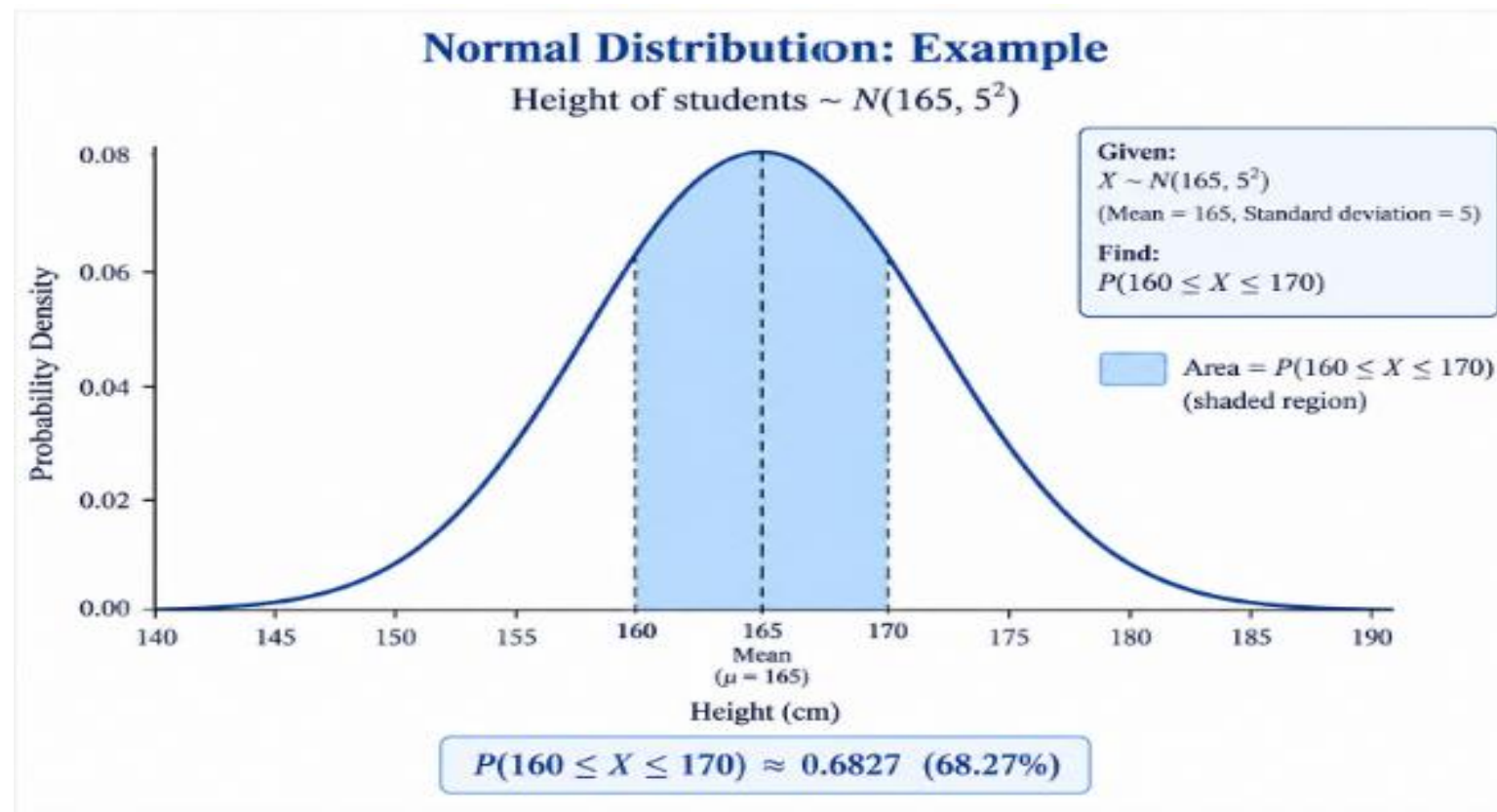
Important: In a continuous distribution, the probability of obtaining one exact value is essentially zero:

$$P(X = 165) = 0$$

Instead, we calculate the probability over an **interval**, such as:

$$P(160 < X < 170)$$

So, area under the probability density curve represents probability.



Normal Distribution:

The normal distribution is a continuous probability distribution with a symmetrical, bell-shaped curve where data centers around a central value with no bias to the left or right. In this distribution, the mean, median, and mode are equal, and the domain extends from $-\infty$ to $+\infty$. It is characterized by two parameters: mean (μ) and variance (σ^2) or standard deviation (σ). A standard normal distribution uses a mean of 0 and a standard deviation of 1.

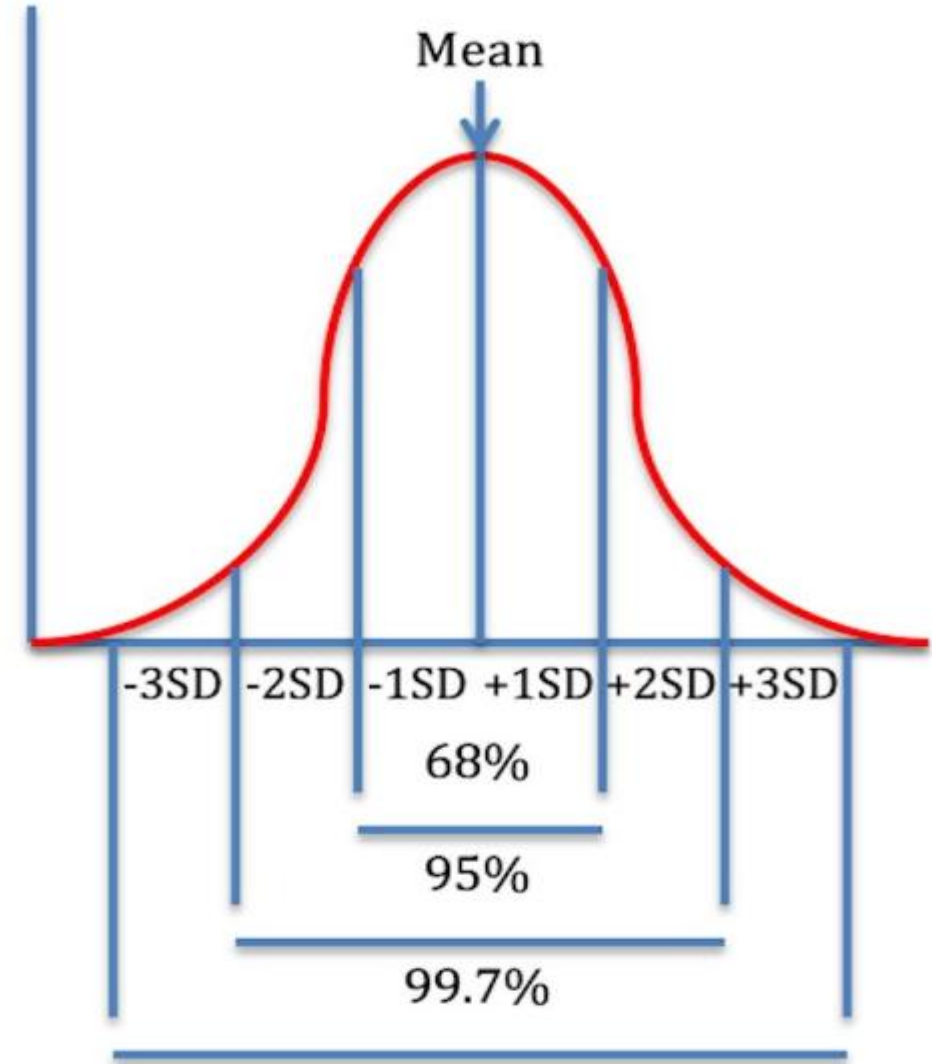
- **Probability Density Function (PDF):**

$$f(x, \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- **Z-Score Formula:** Used to normalize data by measuring how many standard deviations a value x is from the mean:

$$z = \frac{x - \mu}{\sigma}$$

Examples: Student heights, population blood pressure, and class marks. Visualized via Q-Q plots by comparing data quantiles/CDFs against a normal distribution.



Rectangular Distribution Overview

Also known as the continuous uniform distribution, a rectangular distribution is a symmetric probability distribution where all intervals of equal length within the distribution's range have an equal probability of occurring. Its probability density function (PDF) forms a rectangle, bounded by a minimum value a and a maximum value b .

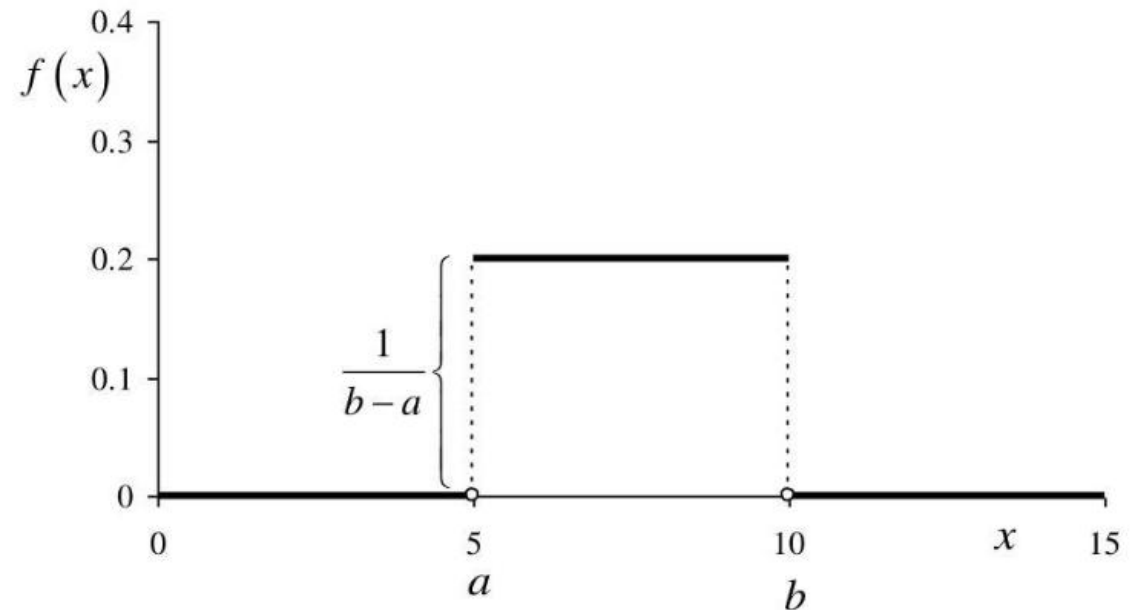
- **Probability Density Function (PDF):**

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

Mean (Expected Value): $\mu = \frac{a+b}{2}$

Variance: $\sigma^2 = \frac{(b-a)^2}{12}$

Commonly used in simulation modeling, random number generation, and scenarios where outcomes are equally likely across a bounded range (such as ideal die rolls or rounding errors).



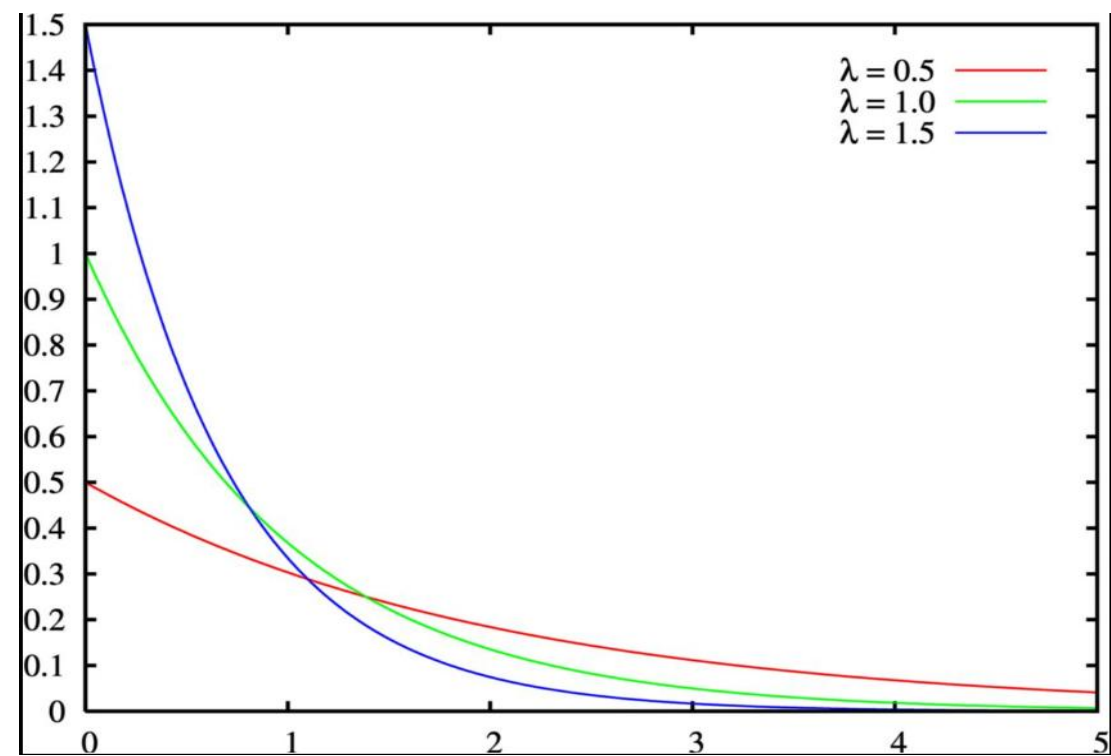
Exponential Distribution:

A continuous probability distribution used to model the time or space between Poisson point process events (events that occur continuously and independently at a constant average rate).

It is **memoryless**, meaning the probability of an event occurring in a future time interval depends only on the duration of that interval, not on how much time has already elapsed.

Probability Density Function (PDF):

$$f(x; \lambda) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x \geq 0 \\ 0 & \text{for } x < 0 \end{cases}$$



Discrete Distribution:

A **discrete distribution** is a probability distribution that describes a random variable whose possible outcomes are distinct, separate values (such as integers like 0, 1, 2, 3...) rather than a continuous range.

Unlike continuous variables (like height or weight) which use a Probability Density Function (PDF), discrete variables rely on two foundational mathematical functions:

- **Probability Mass Function (PMF):** Denoted as $P(X=x)$ or $p(x)$, it gives the exact probability that a discrete random variable X equals a specific value x .
- **Cumulative Distribution Function (CDF):** Denoted as $F(x)=P(X\leq x)$, it gives the sum of probabilities for all values up to and including x .

Rules

Non-negativity: For every possible value x , $P(X=x)\geq 0$.

Total Probability: The sum of all individual probabilities across the entire sample space equals 1:

Binomial Distribution

Binomial distribution is another distribution that is often encountered in machine learning. It has only two outcomes: success or failure. This is also called a Bernoulli trial.

The objective of this distribution is to find the probability of getting k successes out of n trials.

The number of ways to choose k successes out of n trials is given by:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Where p is the probability of success and $1 - p$ (or q) is the probability of failure:

$$p^k(1-p)^{n-k} \quad \text{or} \quad p^k q^{n-k}$$

Probability Mass Function (PMF)

Combining both gives the probability function for a binomial distribution

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Mean (μ): $\mu = n \times p$

Variance (σ^2): $\sigma^2 = np(1 - p)$

Standard Deviation (σ): $\sigma = \sqrt{np(1 - p)}$

Poisson Distribution :

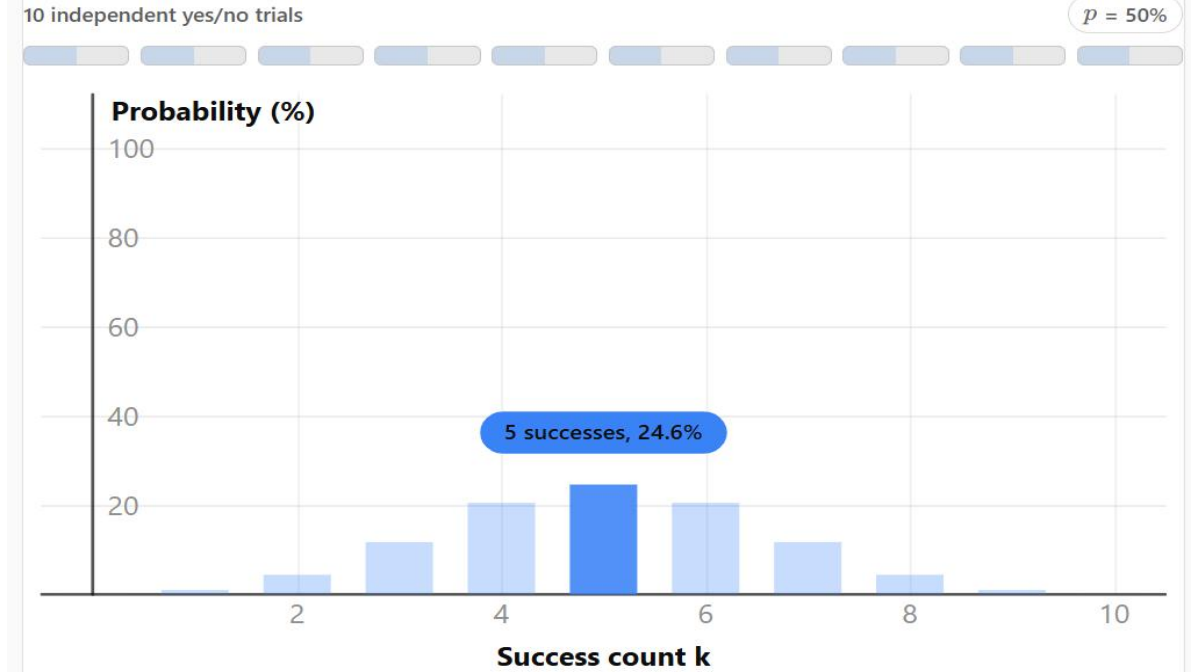
It is another important distribution that is quite useful. Given an interval of time, this distribution is used to model the probability of a given number of events k . The mean rate λ is inclusive of previous events. Some of the examples of Poisson distribution are number of emails received, number of customers visiting a shop and the number of phone calls received by the office.

The PDF of Poisson distribution is given as follows:

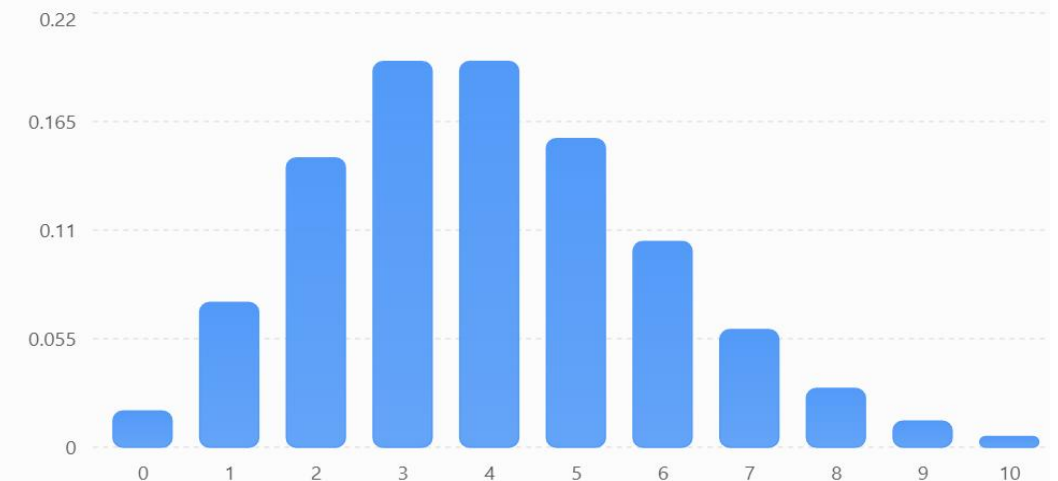
$$f(X = x; \lambda) = \Pr[X = x] = \frac{e^{-\lambda} \lambda^x}{x!}$$

Here, x is the number of times the event occurs and λ is the mean number of times an event occurs.

The mean is the population mean at number of emails received and the standard deviation is $\sqrt{\lambda}$



Example with average event rate $\lambda = 4$.



Bernoulli Distribution:

This distribution models an experiment whose outcome is binary. The outcome is positive with p and negative with $1 - p$. The PMF of this distribution is given as:

$$f(k; p) = \begin{cases} q = 1 - p & \text{if } k = 0 \\ p & \text{if } k = 1. \end{cases}$$

The mean is p and variance is $p(1 - p) = q$

Example with probability of success $p = 0.5$.



Density Estimation

Suppose we have observed data values $x_1, x_2, \dots, x_n$

from a larger population whose probability distribution is unknown. Density estimation is the process of estimating the underlying probability density function (PDF) from these observed data values.

The estimated density function is represented by $\hat{p}(x)$

A common anomaly-detection rule is:

$$p(x_i) < \varepsilon$$

where ε is a predefined threshold.

Thus:

- If $p(x_i) < \varepsilon \rightarrow$ Anomaly / Outlier
- If $p(x_i) \geq \varepsilon \rightarrow$ Normal observation

There are two main approaches:

Type	Basic idea	Distribution assumption
Parametric Density Estimation	Assume a particular probability distribution and estimate its parameters	Distribution is assumed known
Non-Parametric Density Estimation	Estimate the density directly from the observed data	No specific distribution is assumed

Density estimation estimates how frequently or likely different data values are, and very low-density values can be identified as anomalies.

There are two main types:

1. Parametric Density Estimation

- Assumes that the data follows a particular distribution.
- Example: Gaussian (Normal) distribution.
- Parameters such as mean and variance are estimated from the data.

2. Non-Parametric Density Estimation

- Does not assume a specific distribution.
- Estimates the density directly from the observed data.
- Examples: Histogram and Kernel Density Estimation (KDE).

2.10 FEATURE ENGINEERING AND DIMENSIONALLY REDUCTION TECHNIQUES

PCA and LDA algorithm and problems

Basic Learning Theory:

- Design of Learning System
- Introduction to Concept of Learning
- Induction biases
- Modelling in Machine Learning
- learning Frameworks

Design of Learning System:

Learning is the process of gaining knowledge and skills through study, experience, or teaching. In Machine Learning (ML), we try to make computers learn from experience/data, similar to how humans learn.

For a computer to learn effectively, the problem should be well-defined (well-posed).

A learning problem consists of three important components:

1. Task (T) – What the computer has to learn or do.
2. Performance Measure (P) – How we measure whether the computer is performing well.
3. Experience (E) – The data or previous examples from which the computer learns.

Tom Mitchell's Definition of Learning

According to Tom Mitchell:

A computer program is said to learn from experience (E) with respect to a task (T) and a performance measure (P) if its performance at task T, as measured by P, improves with experience E.

Component

Example

Task (T)

Identify whether an email is spam or not

Performance (P)

Accuracy of classification

Experience (E)

Previously labeled spam and non-spam emails

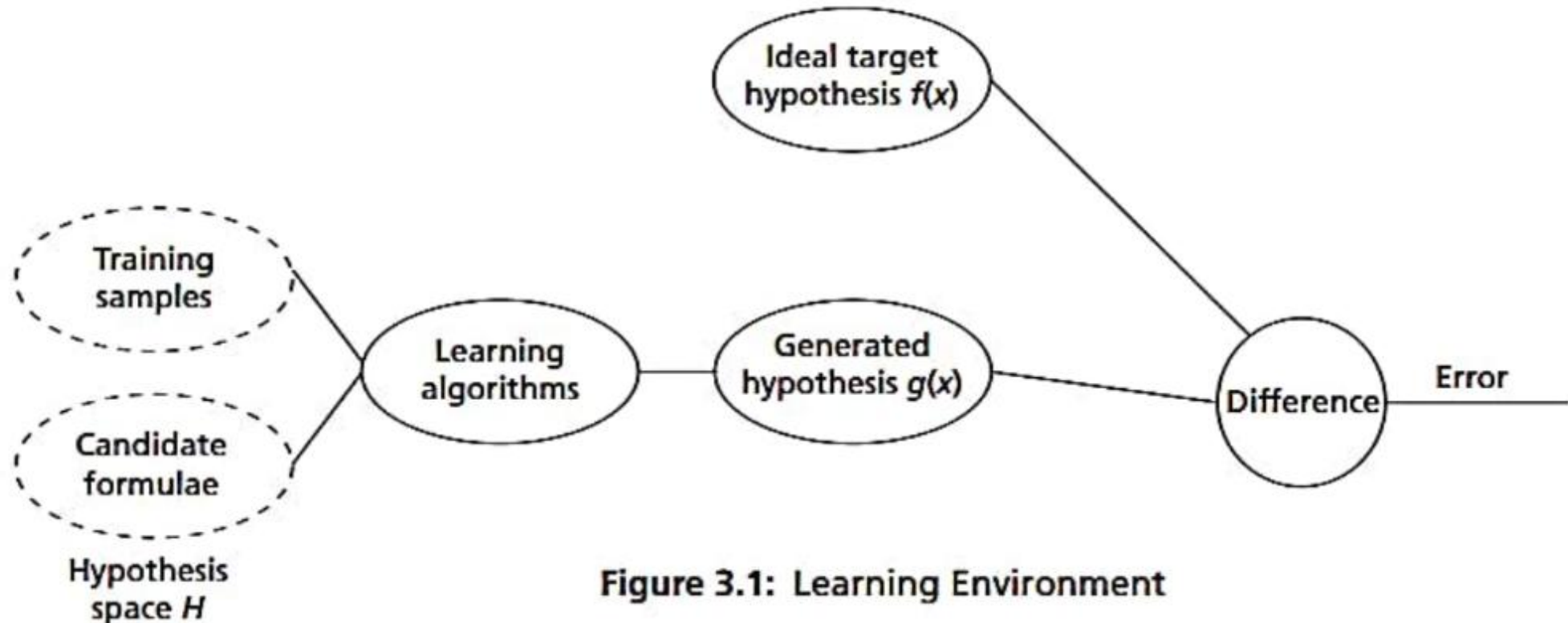


Figure 3.1: Learning Environment

Some important learning types are:

No.	Learning Type	Simple Meaning	Example
1	Learning by Memorization (Rote Learning)	Learning by remembering information through repetition, without necessarily understanding the concept.	Memorizing multiplication tables.
2	Learning by Examples (Inductive Learning)	Learning a general rule or concept by observing several examples or past experiences.	Seeing many spam emails and learning what characteristics indicate spam.
3	Learning by Being Taught	Learning from a teacher or expert. It can be passive or active, where the learner asks questions to obtain labels or information.	A teacher explaining how a decision tree works.
4	Learning by Critical Thinking (Deductive Learning)	Using known facts and information to derive a new fact or conclusion.	If all humans are mortal and Ravi is human, concluding that Ravi is mortal.
5	Self-Learning (Reinforcement Learning)	Learning independently through rewards and punishments based on actions and mistakes.	A robot learns to navigate a room by receiving rewards for reaching the target.
6	Learning to Solve Problems (Cognitive Learning)	Learning by developing a method or sequence of steps to achieve a particular goal.	Finding the shortest route from one city to another.
7	Learning by Generalizing Explanations (Explanation-Based Learning – EBL)	Using expert/domain knowledge to understand examples and improve the learning of concepts.	Using a medical expert's knowledge to improve a disease-classification model.

Design of Learning System

A **learning system** is a system designed around a learning algorithm so that it can learn from experience and improve its performance.

The design of a learning system generally involves **four main steps**:

- 1. Choosing a Training Experience**
- 2. Choosing a Target Function**
- 3. Representing the Target Function**
- 4. Function Approximation**

1. Choosing a Training Experience

The first step is deciding **what experience or data the system will learn from.**

Example: Chess Game

Suppose we want to build a machine learning system that learns to play chess.

There are two ways to provide training experience:

Direct experience

- The system is directly given chess board positions and the correct moves.
- Example:

Board position → Best move

Indirect experience

- The system is given the sequence of moves and the final result of the game.
- The system learns which moves were good or bad based on the result.

For example:

Game → Win → Learn which moves contributed to winning.

If a supervisor/expert is available, the expert can label the **correct moves**.

If there is no supervisor, the chess program can **play against itself** and learn from the results.

2. Choosing the Target Function

The target function specifies what the learning system should learn.

In simple words:

Target function = The desired output that the learning system tries to predict.

Chess Example

The system needs to decide:

"Is this a good move or a bad move?"

3. Determining the Target Function Representation

After deciding what to learn, we must decide how to represent the knowledge.

The target function can be represented using:

- Tables
- Rules
- Decision trees
- Neural networks
- Mathematical equations

Example: Linear Representation

Suppose the chess board has three important features:

- x_1 = number of useful pieces
- x_2 = position of pieces
- x_3 = control of important squares

4. Function Approximation

The actual target function may be very complex and may not be known exactly.

Therefore, the learning algorithm tries to approximate the target function using the available training data.

For example:

Training data

Board features $\rightarrow$ Good/Bad move

The learning algorithm studies these examples and learns an approximate function:

Features $\rightarrow$ Predicted value

Concept learning

Concept learning is an important topic in Machine Learning. It refers to the problem of inferring a Boolean-valued function from training examples.

“Concept learning is the task of learning a general rule (concept) from specific examples.”

OR

“A task of acquiring potential hypothesis(solution) that best fits the given training example.”

A **concept** is a function that maps an input instance to either:

- **Positive (Yes / True)**
- **Negative (No / False)**

$$c : X \rightarrow \{0, 1\}$$

Instance/sample:

An individual example described by attributes.

Example (Weather dataset): **Each row = Instance**

Sky	Temp	Humidity	Wind	Play
Sunny	Warm	Normal	Strong	Yes
Rainy	Cold	High	Strong	No

Attribute

A property used to describe an instance.

Example:

- Sky
- Temperature
- Humidity
- Wind

Hypothesis

A candidate rule that classifies instances as positive or negative.

Example:

If Sky = Sunny AND Temp = Warm $\rightarrow$ Play = Yes

Hypothesis Space (H)

The set of all possible hypotheses that can be formed using.

- Specific values
- General value (?)
- No value ($\emptyset$)

Then the hypothesis space contains all combinations.

Goal of Concept Learning

Given:

- A set of training examples
- Each labeled as Positive or Negative

Find a hypothesis h such that:

$$h(x)=c(x)$$

for all training examples

That means:

The hypothesis should correctly classify all training data.

Example: Suppose we want to learn the concept: *Days suitable for playing tennis*''

Sky	Temp	Play
Sunny	Warm	Yes
Rainy	Cold	No
Sunny	Cold	No
Sunny	Warm	Yes

The learning task is to find a rule like:

If Sky = Sunny AND Temp = Warm → Yes
Else → No

Concept learning can be viewed as a **search problem**

The search space = Hypothesis Space (H)

Because:

- There are many possible hypotheses in H.
- We must search through H
- To find the hypothesis that best fits the training data.

Hypotheses can be ordered from:

- Most General (accepts all instances) $\langle ?, ? \rangle$
- Most Specific (rejects all instances) $\langle \emptyset, \emptyset \rangle$

The Version Space is the subset of hypotheses that are:

✓ Consistent with all training examples.

It lies between:

- **S boundary** → Most specific hypothesis
- **G boundary** → Most general hypothesis

Searching the Hypothesis Space

1. General-to-Specific Search: Start with most general hypothesis, gradually specialize it. Used in
 - **Find-S Algorithm**
 - **Candidate Elimination Algorithm**
1. Specific-to-General Search: Start with most specific hypothesis, gradually generalize it.

Find the following for the given example:

Consider

Feature1 $\rightarrow$ A, B

Feature2 $\rightarrow$ X, Y

Instance space = $(A, X) \cup (A, Y) \cup (B, X) \cup (B, Y)$

- **Hypothesis Space:** $(A, X), (A, Y), (A, \emptyset), (A, ?), (B, X), (B, Y), (B, \emptyset), (B, ?), (\emptyset, X), (\emptyset, Y), (\emptyset, \emptyset), (\emptyset, ?), (?, X), (?, Y), (?, \emptyset), (?, ?)$ - **16 Hypothesis**

Semantically Distinct Hypothesis : $(A, X), (A, Y), (A, ?), (B, X), (B, Y), (B, ?), (?, X), (?, Y), (?, ?), (\emptyset, \emptyset)$ - **10**

Find-S Algorithm

Example	Sky	Temp	Humidity	Wind	Play
1	Sunny	Warm	Normal	Strong	Yes
2	Sunny	Warm	High	Strong	Yes
3	Rainy	Cold	High	Strong	No
4	Sunny	Warm	Normal	Weak	Yes

1. Start with the **most specific hypothesis:**

$S_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$

This means: Reject all instances initially.

1. Instance1:

$\langle \text{Sunny, Warm, Normal, Strong} \rangle$

Since hypothesis is most specific,

replace

$\emptyset$ with attribute values.

$\langle S_1 = \langle \text{Sunny, Warm, Normal, Strong} \rangle$

1. $\langle \text{Sunny, Warm, High, Strong} \rangle$

Updated hypothesis:

$\langle S_2 = \langle \text{Sunny, Warm, ?, Strong} \rangle$

4. $\langle \text{Rainy, Cold, High, Strong} \rangle$ is a negative sample so Find-s algorithm ignores

$\langle S_3 = \langle \text{Sunny, Warm, ?, Strong} \rangle$

5. $\langle \text{Sunny, Warm, Normal, Weak} \rangle$

Updated hypothesis:

$\langle S_4 = \langle \text{Sunny, Warm, ?, ?} \rangle$

6. **$S_{\text{final}} = \langle \text{Sunny, Warm, ?, ?} \rangle$**

The learned concept is:

If Sky = Sunny AND Temperature = Warm $\rightarrow$ Play = Yes
Humidity and Wind do not matter.

Candidate Elimination Algorithm

The Candidate Elimination Algorithm is an advanced concept learning method that finds all hypotheses consistent with the training data, not just one.

It works by maintaining the Version Space — the set of all consistent hypotheses.

Unlike Find-S (which finds only one hypothesis),
Candidate Elimination:

- ✓ Uses both positive and negative examples
 - ✓ Maintains two boundaries:
 - S (Specific boundary) → Most specific consistent hypothesis
 - G (General boundary) → Most general consistent hypothesis
- Eliminates inconsistent hypotheses step by step

1. Initialize:

- S = most specific hypothesis
- G = most general hypothesis

2. For each training example:

- If Positive:
 - Remove inconsistent hypotheses from G
 - Generalize S minimally
- If Negative:
 - Remove inconsistent hypotheses from S
 - Specialize G minimally

3. Output S and G

Candidate elimination algorithm

Most Specific Hypothesis:

$S_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Most General Hypothesis:

$G_0 = \langle ?, ?, ?, ? \rangle$

1. **Instance-1:** $\langle \text{Sunny, Warm, Normal, Strong} \rangle$

$S_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Replace $\emptyset$ with values:

$S_1 = \langle \text{Sunny, Warm, Normal, Strong} \rangle$

$G_1 = \langle ?, ?, ?, ? \rangle$ no change

2. **Instance-2:** $\langle \text{Sunny, Warm, High, Strong} \rangle$

Compare with S_1 :

$\langle S_2 = \langle \text{Sunny, Warm, ?, Strong} \rangle$

Remove hypotheses inconsistent with this positive example.

Current G still consistent $\rightarrow$ No change.

$G_2 = \langle ?, ?, ?, ? \rangle$

Example

1

Sky

Temp

Humidity

Wind

Play

Sunny

Warm

Normal

Strong

Yes

2

Sunny

Warm

High

Strong

Yes

3

Rainy

Cold

High

Strong

No

4

Sunny

Warm

Normal

Weak

Yes

Instance-3: $\langle \text{Rainy, Cold, High, Strong} \rangle$

Now we update G , not S .

We must specialize G so it does NOT cover this negative example.

Current: $G_2 = \langle ?, ?, ?, ? \rangle$

To exclude $\langle \text{Rainy, Cold, High, Strong} \rangle$, we specialize attributes one by one: Possible minimal specializations:

$\langle \text{Sunny, ?, ?, ?} \rangle$

$\langle ?, \text{Warm, ?, ?} \rangle$

$\langle ?, ?, \text{Normal, ?} \rangle$

$\langle ?, ?, ?, \text{Weak} \rangle$

Problem-1

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

$S_0:$ $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

$S_1:$ $\langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle$

$S_2:$ $S_3:$ $\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$

$S_4:$ $\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$

$G_4:$ $\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$ $\langle ?, \text{Warm}, ?, ?, ?, ? \rangle$

$G_3:$ $\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$ $\langle ?, \text{Warm}, ?, ?, ?, ? \rangle$ $\langle ?, ?, \text{Normal}, ?, ?, ? \rangle$ $\langle ?, ?, ?, ?, \text{Cool}, ? \rangle$ $\langle ?, ?, ?, ?, ?, \text{Same} \rangle$

$G_0:$ $G_1:$ $G_2:$ $\langle ?, ?, ?, ?, ?, ? \rangle$

S

⟨Sunny, Warm, ?, Strong, ?, ?⟩

⟨Sunny, ?, ?, Strong, ?, ?⟩

⟨Sunny, Warm, ?, ?, ?, ?⟩

⟨?, Warm, ?, Strong, ?, ?⟩

G

⟨Sunny, ?, ?, ?, ?, ?⟩

⟨?, Warm, ?, ?, ?, ?⟩

Induction biases

Induction is a process of learning a target function or generalizing training data into a general model.

Inductive bias is the set of prior assumptions considered by a learning algorithm beyond the training data, in order to perform induction. It is also called the *bias of the algorithm* and is similar to prior knowledge used for learning new concepts.

1. Bias

Bias is the error caused when a model is **too simple** and makes strong assumptions about the data.

- High bias → **Underfitting**
- The model performs poorly on both training and test data.
- Example: Using a **straight line** (linear regression) to model a highly complex curved relationship.

Bias is the error due to a model being too simple to capture the actual pattern in the data.

2. Variance

Variance is the error caused when a model is too sensitive to the training data.

- High variance → **Overfitting**
- The model performs very well on training data but poorly on unseen/test data.
- Example: A very deep decision tree that memorizes the training examples.

Variance is the error due to a model changing too much when the training data changes.

Bias variance tradeoff

Table 3.3: Tradeoff between Bias and Variance

Bias	Variance
Low Bias algorithms have less simplifying assumptions. Machine learning models: Decision Trees, k-Nearest Neighbors (k-NN) and Support Vector Machines (SVM)	Low variance occurs when training data is general. But low variance algorithms have a slight difference in the estimated target function when training dataset changes. Machine learning models: Linear Regression, Linear Discriminant Analysis (LDA) and Logistic Regression
High Bias algorithms have more simplifying assumptions or considers the features that are not relevant. They underfit with the training data. Machine learning models: Parametric algorithms, Linear algorithms – Linear Regression, Linear Discriminant Analysis and Logistic Regression, Naïve Bayes algorithm	High variance occurs when training data is more sensitive to outliers and noise. High variance algorithms have more difference in the estimated target function when the training dataset changes. Machine learning models: Non-linear algorithms - Decision Trees, k-Nearest Neighbors and Support Vector Machines, Non-parametric algorithms
Linear algorithms are fast learning due to simplifying assumptions and hence exhibit high bias.	Decision tree algorithms have high variance, when the tree is grown very deep and they are not pruned before use.
High bias algorithms are consistent but inaccurate on average.	High variance algorithms are accurate on average but are inconsistent.

Modelling in Machine Learning

- I. Model selection and model evaluation
- II. Resampling methods

The main resampling methods are:

- 1. Holdout Method**
- 2. Random Subsampling**
- 3. Cross-Validation**
 - k-Fold Cross-Validation
 - Stratified k-Fold Cross-Validation
 - Leave-One-Out Cross-Validation (LOOCV)
 - Leave-P-Out Cross-Validation
- 4. Bootstrap Sampling**
- 5. Repeated Cross-Validation**

learning Frameworks

A learning framework is a systematic process of collecting data, preparing it, learning patterns using an algorithm, evaluating the learned model, and using it to make predictions.

END OF UNIT-2